Extending AI-Agent Incident Analysis to Multi-Agent Systems

AFACS, an HFACS-based framework, and an audit of public traces

James Le Cuirot
Senior First Officer and Flight Safety Officer · MSc Human Factors in Aviation

Draft, 3 November 2026

Abstract

Ezell, Roberts-Gaal and Chan (2025) proposed a framework for analysing incidents involving AI agents, with system factors, contextual factors and cognitive errors, and listed open challenges for future work. Their framework treats single agents, has no category for deliberate departures from known requirements, and has not been applied to real traces. This report extends it to multi-agent systems using the structure of the Human Factors Analysis and Classification System (HFACS) from aviation. The extension, AFACS, adds a supervision and orchestration tier, a coordination category analogous to crew resource management, and violations coded from behaviour with a graded statement of the evidence (stated, inferred or thin); it also proposes a model-development code that HFACS lacks. A cross-walk maps each of Ezell et al.'s factors onto the AFACS layers. The framework is then tested against the public MAST dataset of multi-agent traces. An audit of that dataset finds that its 1,642 rows hold 1,537 distinct traces, that one group of 100 rows contains only 30 distinct texts with unrelated labels, and that MAST's LLM judge agrees with itself at a mean per-mode Cohen's κ of 0.43 on 30 traces it labelled twice. Audited against the eighteen kinds of information Ezell et al. say investigators need, every framework's logs record what the agents said and did, but none includes the system documentation, artefacts or access needed to reconstruct a failure; sampling settings are logged by one framework and seeds by none, and the only logged model identifier contradicts MAST's metadata for one group of 30 traces. In an AI pilot, Claude coded 250 MAST-failed traces three times (two runs of one model and one run of a second). The layers were applied consistently (Cohen's κ 0.63–0.76 between models and 0.74–0.87 between runs; individual codes 0.52 and 0.59), well above the agreement of MAST's judge with itself. Supervision factors appear in about two-thirds of failures and coordination factors in two-fifths. Violations appear in 9–13%, are mostly inferred from the act rather than stated, and are concentrated in two frameworks; MAST's "Disobey Task Specification" mixes them with errors. The annotator's quotes were genuine (over 99.5% found verbatim). Its design-level codes are inferences from traces: checked against the frameworks' documentation and code, its design claims usually identified a real weakness but overstated it as an absence (8 of 24 confirmed in full, 13 in part, 2 contradicted), and the check also exposed discrepancies in MAST's own descriptions of the systems.

1 Introduction

When an aircraft accident is investigated, "pilot error" is where the analysis starts. Reason's model of organisational accidents describes how the unsafe act at the sharp end is made likely by latent conditions created earlier and higher up: preconditions in the immediate situation, supervision, and organisational decisions [1, 2]. The Human Factors Analysis and Classification System (HFACS) turned that model into a coding scheme with four tiers, from organisational influences down to unsafe acts [3, 4]. HFACS also keeps Reason's distinction between errors, which are unintended (slips, lapses and mistakes), and violations, which are departures from known rules. The two call for different remedies, and survey evidence suggests they are different kinds of behaviour with different correlates [5].

LLM agents are language models that act over many steps: they plan, call tools, write and run code, and exchange messages. A multi-agent system assigns several such agents roles, such as coder, tester and orchestrator, much as a crew divides duties. Ezell, Roberts-Gaal and Chan [6] have proposed the first framework for analysing incidents involving AI agents in this tradition. It explains incidents through three categories of factor, system factors, contextual factors and cognitive errors, linked in a causal chain, and it sets out what information investigators need to tell them apart. Its authors close with a table of open challenges for future work.

Three features of that framework leave room for extension. It treats a single agent, so it has no place for an orchestrator that supervises other agents, nor for breakdowns between agents. Its agent-level category contains errors only: the paper cites an agent that reasoned about cheating its unit tests before doing so [7], but no category records a deliberate departure from a known requirement. And it is conceptual: it has not been applied to real traces.

This report extends the framework to multi-agent systems using the structure of HFACS, and calls the result AFACS. It takes up five of the thirteen open challenges in Ezell et al.'s Appendix B: *non-exhaustive coverage*, since multi-agent systems are a domain whose specific factors their taxonomy does not cover; *emerging factors*, by adding violations for the departures they mention but do not categorise; *high-level cognitive errors*, by testing their error categories against HFACS's on real traces; *reconstruction difficulties* and the related problem of *limited data access*, by auditing what openly released logs allow an investigator to reconstruct; and, in part, *expertise shortages*, through a codebook with explicit decision rules and a test of whether AI-assisted coding is consistent. Their remaining challenges (reporting infrastructure, accountability mechanisms, sensitive data, the legal landscape, data retention and voluntary adoption) are institutional and outside the scope of a trace study.

The study uses the public MAST dataset of multi-agent traces [8] and asks four research questions.

- **RQ1, cross-walk.** How do Ezell et al.'s factors map onto the HFACS layers, and what does a multi-agent setting add?
- **RQ2, information adequacy.** Do real multi-agent traces contain the information Ezell et al. say investigators need?
- **RQ3, applicability.** Can the extended framework be applied consistently, and how often do failures involve the added supervision, coordination and violation categories?
- **RQ4, flat labels.** How do MAST's fourteen flat labels map onto the extended framework?

The contributions are: (i) AFACS, an extension of Ezell et al.'s framework to multi-agent systems with a supervision tier, a coordination category and evidence-graded violations, together with an item-by-item cross-walk; (ii) an audit of the MAST dataset documenting duplicates, corrupt and misfiled rows, logging artefacts and the test–retest reliability of its LLM judge; (iii) an audit of MAST's logs against Ezell et al.'s information requirements, showing what public multi-agent traces do and do not let an investigator establish; and (iv) an AI-coded pilot on 250 traces, coded three times, measuring how consistently the framework can be applied and what it shows that the flat labels do not. Human validation with independent coders is planned as a second study, for which a blinded, pre-registered calibration set has already been frozen.

2 Background and Related Work

2.1 Incident analysis for AI agents

Ezell et al. define three categories of factor [6]. System factors are static aspects of an agent's development: training data, feedback, learning methods, the system prompt and scaffolding code. Contextual factors are features of the situation: the information available, the tools, and how the task was defined. Cognitive errors are failures of observation, understanding, decision-making and action execution, which are the most proximate to the incident but arise because of system and contextual factors. The paper also lists eighteen items of information, in three groups (activity logs, system documentation and access, and tool information), that investigators need in order to decide which factors apply (their Table 1). It acknowledges that its treatment of cognitive errors is "necessarily high-level" because the cognitive science of AI agents is nascent, and that its factors are not exhaustive. Shamsujjoha et al. use the Swiss-cheese idea to design layered runtime guardrails for agents [9], and Mylius applies STPA, a system-theoretic hazard analysis, to a frontier-AI scenario [10, 11]. Both look forward, at defences and hazards. Ezell et al. and AFACS look back, at what happened.

2.2 Agent failure taxonomies

MAST was derived bottom-up from 150 traces and refined over three rounds; its authors report human inter-annotator agreement of $\kappa = 0.88$ in the final round, $\kappa = 0.79$ on systems not seen during develop-

ment, and $\kappa = 0.77$ between an o1-based LLM annotator and human experts [8]. Its discussion draws on organisational safety research, but the taxonomy itself has no causal layers and no error/violation distinction. Other taxonomies classify errors by type, finding that the best LLM locates only 11% of human-annotated errors [12]; by the component that broke [13]; by the agent and step responsible, where the decisive step is found 14.2% of the time [14]; or by the side that should own the fix, with an LLM judge reaching $\kappa = 0.76$ against human labels [15]. These works answer "what broke" and "who and when". AFACS adds "what kind of act" and "what latent conditions".

2.3 HFACS and its reliability

The closest methodological precedent is Wiegmann and Shappell [16]: an aviation psychologist and a commercially rated pilot independently re-coded 319 causal factors from 119 NTSB accident reports into HFACS and agreed at $\kappa = 0.71$. HFACS reliability is contested. Olsen and Shorrock found low agreement, between and within coders, for a modified HFACS used in air traffic control [17]. A review by Cohen, Wiegmann and Shappell found reliability higher with coder training, larger samples and fewer departures from the original framework [18]; that review drew a published critique [19]. LLMs applied to HFACS coding of aviation reports reach far lower exact-match than partial-match agreement [20]. AFACS therefore keeps the original four tiers, and agreement is reported at layer level as well as code level.

2.4 AI behaviours that resemble violations

Specification gaming, in which a system satisfies the letter of its objective while defeating its purpose, is the standard AI-safety term for many behaviours AFACS would code as exceptional violations [21]. ImpossibleBench builds coding tasks in which passing requires breaking the specification, so any pass cannot be accidental [22]. Baker et al. show that reasoning traces can reveal reward hacking, and that models trained against such monitors learn to hide it [7]. In constructed scenarios, frontier models reason explicitly about the constraints they face and choose to circumvent them [23, 24]. OverclaimBench measures false claims of task completion from transcripts alone, with a criterion that "requires no inference about intent" [25], the same behavioural stance as AFACS. At the aggregate level, Hägele et al. find that failures look less like coherent pursuit of unintended goals and more like an incoherent "hot mess" as tasks lengthen [26], a reminder that most failures are likely to be errors.

3 The AFACS Framework and its Cross-Walk (RQ1)

Each failed trace is coded at every layer where the trace gives evidence (multi-label), with a deepest contributing layer and a confidence rating (high, medium or low). Table 1 lists the layers and codes of codebook v1 (`codebook/v1.md`). The codebook is the author's.

3.1 Adapting HFACS to multi-agent runs

Five decisions adapt HFACS to multi-agent benchmark runs. First, the humans whose decisions shape a run are placed in the existing tiers rather than in a new one. The user who sets the task sits at the top of the supervision chain, so a deficient task is coded L3.4 whether a human user or an orchestrating agent set it, and the coder records which. The people who launch and configure a run are treated as part of the organisation: when the run itself was set up wrongly, for instance launched with an empty task, AFACS codes `L4.5` Operation set-up, the analogue of a defective dispatch, rather than excluding the trace. Second, when the framework's own software (not the model) misbehaves, the cause decides the code: a malfunction during the run is a precondition (L2.1), a limitation built into the design is L4.1 or L4.2. Third, a failure need not include an unsafe act. If a tool or service fails and the agents respond reasonably, for example by reporting the problem rather than inventing a result, the failure is coded at the precondition or design layer and no L1 code is given. HFACS assumes an unsafe act at the sharp end of every accident; forcing one here would misrepresent the failure. Fourth, a loop is coded as the repeating agent's unsafe act and, where a supervising agent could have broken it, also as a failure to correct a known problem (L3.3). Fifth, an omission such as never verifying a result is coded by whose

Table 1: AFACS layers and codes (codebook v1). L4.4, added from Ezell et al.'s system factors, needs developer-side evidence and is not applied from traces (Section 3.3). L4.5 covers the people who launched the run.

Layer	Code	Description
L4 Organisational / design	L4.1	Architecture and role design: role split, missing verifier, no termination logic
	L4.2	Resource provisioning: tools, step or token budgets, context limits
	L4.3	Process: no standard procedure for verification or handover
	L4.4	Model development (developer evidence only): training data, feedback or learning methods
	L4.5	Operation set-up: the run was launched or configured wrongly, e.g. an empty task
L3 Supervision / orchestration	L3.1	Inadequate supervision: no check on subagent output
	L3.2	Planned inappropriate operation: task beyond an agent's capability or information
	L3.3	Failure to correct a known problem: visible loop or error, no intervention
	L3.4	Deficient specification: ambiguous, contradictory or incomplete task or prompt, set by the user or by an orchestrating agent
L2 Preconditions	L2.1	Environment: tool or API failure, bad or unavailable information
	L2.2	Agent state: context saturation, lost instructions, carried-over error
	L2.3	Coordination: communication breakdown, withheld or ignored input
L1 Unsafe acts: errors	L1.E1	Skill-based: execution slip, e.g. a malformed tool call
	L1.E2	Decision: wrong plan or choice given adequate information
	L1.E3	Perceptual: misread observation, tool output or message
L1 Unsafe acts: violations	L1.V1	Routine: habitual shortcutting across the trace
	L1.V2	Exceptional: one-off departure, e.g. fabricating a result or editing tests
	L1.V?	Indeterminate: the criteria appear to hold but the evidence is thin

Table 2: Condensed cross-walk from Ezell et al.'s factors to AFACS codes. The last three rows have no counterpart in Ezell et al. †Needs developer evidence; not applied from traces (Section 3.3).

Ezell et al. factor	AFACS code	Note
System: training data, feedback, learning methods	L4.4†	Latent; feedback that rewards appearing successful is a typical condition behind violations
System: system prompt underspecified or encouraging undesirable behaviour	L3.4	System prompt treated as part of the specification
System: scaffolding not working as intended	L4.1/L4.2 or L2.1	Coded by cause: built into the design, or a failure during the run
Contextual: information inaccessible, missing, poor or malicious	L2.1	
Contextual: tools unusable or failing; tool errors not visible	L2.1 or L4.2	Run-time failure, or provisioning by design
Contextual: unclear or conflicting task; resource limits	L3.4; L4.2	
Cognitive: observation (failure to notice, wrong expectation)	L1.E3	Memory capacity at L2.2
Cognitive: understanding (misinterpretation; conclusions from poor information)	L1.E3; L1.E2	With L2.1 where the information was poor
Cognitive: decision-making (goal priority, non-viable plans, criteria)	L1.E2	Distraction from the goal at L2.2
Cognitive: action execution (incorrect execution; no action; no error monitoring)	L1.E1; L1.E1/E2; L1.E3 or L3.3	L3.3 where a supervising agent should have caught it
(none: single-agent framework)	L3.1–L3.3	Orchestrator fails to check, assigns beyond capability, or leaves a visible problem uncorrected
(none)	L2.3	Coordination between agents, the crew resource management analogue
(mentioned, not categorised)	L1.V1, L1.V2, L1.V?	Violations with an evidence grade (Section 3.4)

job it was: an L1 act if an agent was told to do it, L3.1 if a supervising agent should have checked, and L4.1 or L4.3 if nobody was given the job.

3.2 Cross-walk to Ezell et al.

Table 2 condenses the item-by-item mapping in `docs/crosswalk_ezell.md`. In broad terms, Ezell et al.'s system factors correspond to L4, with the system prompt treated as part of the task specification at L3.4; their contextual factors correspond to L2.1, with task definition at L3.4 and resource limits at L4.2; and their cognitive errors correspond to the L1 errors, with memory limits and distraction treated as agent state at L2.2. Their four cognitive functions (observation, understanding, decision-making and action execution) cut the same ground as HFACS's three error types differently. AFACS therefore records both for every error: the HFACS type (E1–E3) as the code, and the Ezell function in a separate field. This allows the cross-walk's predictions (E3 with observation and understanding, E2 with decision, E1 with action) to be checked against coded traces, and RQ3 to compare which scheme can be applied more consistently.

The mapping also runs the other way. Ezell et al. name model development as a cause, which HFACS has no tier for; HFACS's nearest category, organisational resource management, covers "human resource management (selection, training, staffing)" [16], and AFACS adds L4.4 by analogy. They name security-related context (malicious inputs, vulnerable tools), which AFACS treats as sub-cases of L2.1. And their list of information requirements gives a standard against which real logs can be audited (RQ2).

Table 3: Evidence needed to code each AFACS layer, and the corresponding information category in Ezell et al. [6].

Layer	Evidence needed	Ezell et al. category
L1 unsafe acts, L2 preconditions	The trace: messages, actions, tool outputs	Activity logs; tool information
L3 supervision	The trace, the task specification and the system prompt	Activity logs (system and user prompts)
L4.1–L4.3 design, process	The trace and what is known of the system's design	System documentation; tool information
L4.5 operation set-up	The trace and what is known of how the run was launched and configured	Activity logs (user prompts); system configuration
L4.4 model development	Developer records: training and reward specifications, change logs, version or checkpoint comparisons	System documentation and change logs; system artefacts; model access

3.3 Where the evidence for each layer comes from

A layered framework needs layered evidence. Each layer can be coded only from evidence an investigator in that position can hold (Table 3). Unsafe acts and preconditions are visible in the trace itself. Supervision failures need the trace together with the task specification and system prompt. Design and process factors need what is known of the system's design. Model development needs records that only the developer holds: training data and reward specifications, change logs, and comparisons between model versions.

For this reason AFACS does not apply L4.4 from traces, and the annotator in this study is never offered it. Developers can test a model-development contribution more directly: by running the same scenario on model versions or checkpoints from before and after a training stage, by checking whether the training signal would have rewarded the behaviour (an exploitable grader that checks only whether tests pass rewards editing the tests [7]), and by running the same targeted behavioural tests on each version. Even then they can usually show that a training choice made a behaviour more likely, not that it caused one particular failure. From traces alone, a model-level contribution can only be looked for across many traces, by holding the framework and benchmark fixed and asking whether a pattern follows the model.

HFACS met the same limit in aviation. When NTSB records were coded into HFACS, supervisory and organisational factors were scarce and "organizational climate" did not appear at all; the authors suggested that such factors may contribute "yet they are rarely identified using existing accident investigation processes" [16]. Organisational causes are established from company records, not from the flight recorder.

3.4 Violations

"Violation" is used in HFACS's sense, applied to behaviour. An investigator never sees inside a pilot's head either: a violation is inferred from evidence that the rule was known and that the action departed from it. AFACS makes the same inference and makes no claim about intent, will or awareness; whether AI agents have intentions is an open question this framework does not try to answer. A failure is coded as a violation only if all three conditions hold: (1) the requirement was in the agent's accessible context; (2) the trace shows the agent represented it, by referencing or acknowledging it, or the deviation is of a kind that cannot plausibly be accidental, such as hardcoding expected test outputs; and (3) the agent proceeded otherwise. If all three hold but the evidence is thin, the code is `L1.V?`; otherwise the failure is an error. In HFACS a routine violation is a habitual departure, often tolerated by supervisors; a single trace can show only that a departure recurs within the run, which is what `L1.V1` means here. Benchmark information the agents never saw cannot satisfy condition (1).

The distinction is recorded because the two call for different remedies. An error (the agent could not comply) is addressed by better tools, clearer tasks, verification or more capable models. A violation (the agent registered the requirement and did otherwise) is not a capability problem; it points to incentives, monitoring and what training rewards. In HFACS, routine violations often have organisational roots, such as pressure or tolerated shortcuts; in an AI agent the corresponding latent condition is often training feedback that rewarded the appearance of success, which AFACS places at L4.4 and which can be evidenced only from the developer's side (Section 3.3).

Table 4: Evidence grades for violations (codebook v1, field `violation_evidence`).

Grade	Use when	Goes with
stated	The agent's own words (reasoning, plan or message) acknowledge the requirement, and it then departs from it	L1.V1, L1.V2
inferred	Nothing in the agent's words shows it registered the requirement, but the act cannot plausibly be accidental	L1.V1, L1.V2
thin	The criteria seem to hold but the evidence for condition (1) or (2) is weak	L1.V?
none	No violation is coded	

Table 5: Data-quality exclusions applied before sampling.

Reason	Rows	Note
Kept	1,505	
Corrupt source group	96	ChatDev–CodeLlama: 100 rows hold 30 distinct texts
Duplicate text	35	identical trajectory seen earlier; first copy kept
No MAST labels	4	all 14 labels null
Misfiled text	2	"HyperAgent" rows that contain ChatDev logs
Total	1,642	

Every coding records the basis for any violation (Table 4), so that readers can see how strong the evidence is and apply their own view on intent. Stated reasoning is evidence, not proof: a model's written reasoning may not reflect what produced its action, and models trained against monitors can learn to hide it [7]. The absence of incriminating reasoning is therefore no proof of an honest error, which is why the *inferred* grade exists.

3.5 Incidents and failures

Ezell et al. analyse incidents: events that cause or could plausibly cause harm. MAST's traces are failed benchmark tasks in which nobody was harmed. Aviation draws the same line and applies the same causal analysis to both: occurrence reporting covers events where no harm resulted, because their causes are the causes of future accidents. AFACS is applied here to failures on that basis.

4 Data and Audit

The data are MAST's public release (`mcemri/MAST-Data`, revision `95118ac`, licence CC-BY-4.0): 1,642 traces from seven multi-agent frameworks (AG2, MetaGPT, ChatDev, Magentic-One, AppWorld, HyperAgent and OpenManus), eight benchmarks and five LLMs. Each record holds one raw log and 14 binary MAST labels assigned by an LLM judge; the authors' released pipeline uses OpenAI's o1 and truncates traces longer than about one million characters before judging. Before building on the labels, the dataset was audited (`docs/data_audit.md`). Table 5 summarises the exclusions.

Duplicates and judge consistency. The 1,642 rows contain 1,537 distinct trajectory texts. Among labelled rows, 65 groups of byte-identical texts cover 166 rows, and in 59 of these groups the copies carry different MAST labels. Thirty Magentic-One traces appear under two group names and were labelled in separate judge runs. Computing Cohen's κ separately for each of the 14 modes and averaging gives 0.43, with raw agreement of 0.80 and only 4 of 30 pairs carrying identical label sets (Figure 1). Per-mode κ ranges from -0.05 (FM-1.2) to 0.67 (FM-2.6). Pooling all modes into one list inflates the figure to 0.54, because modes that are almost always absent count as agreement; this is why per-code averaging is used throughout. In the ChatDev–CodeLlama group, copies of the same text agree at a mean per-mode κ of -0.04 over 111 pairs, which suggests that text and labels were mismatched upstream; the group is excluded.

MAST's authors report that their few-shot o1 judge agrees with human experts at $\kappa = 0.77$ [8]. The two figures are not directly comparable: they cover different traces and may be computed differently.

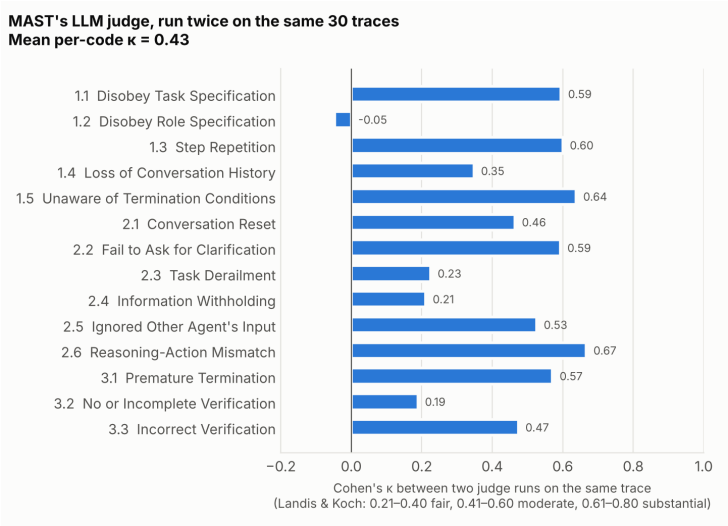


Figure 1: Test–retest agreement of MAST's LLM judge on 30 byte-identical Magentic-One traces labelled in two runs. Cohen's κ per failure mode; mean 0.43.

Even so, repeat runs of the same judge on identical traces agreeing at a mean per-mode κ of 0.43 suggest that individual MAST labels carry substantial run-to-run noise, which matters for any analysis, including RQ4 here, that treats them as a reference.

Metadata errors and logging artefacts. The OpenManus traces are tagged as a programming benchmark but are GAIA questions; the AG2 "GSM" traces are GSM-Plus. Both are corrected. In 29 of the 30 OpenManus logs the task is printed truncated to about 50 characters, although the agents' plans show they received the full text; in the remaining one the task was typed in interactively and never logged. Rendered traces carry a note in both cases. Five traces have an empty task: the harness gave the agents nothing to do. MAST nevertheless labels the MetaGPT example `MetaGPT_ProgramDev2_GPT4o__099` with FM-1.1 Disobey Task Specification and FM-2.2 Fail to Ask for Clarification (Section 7). These artefacts bear directly on Ezell et al.'s reconstruction challenge, and RQ2 examines them systematically.

The human-labelled subset. MAST's separate human-labelled file holds 19 traces (not 21), coded with four different revisions of the taxonomy. Within every round, different traces carry identical three-annotator label vectors, contradicting the original annotation sheets in MAST's repository, so the file is not used.

No ground-truth outcome. MAST records no task success flag, so a "failed" trace here means one the judge flagged with at least one mode. Only 60 traces carry the benchmark's own verdict in the log (AppWorld and the AG2 GSM-Plus subset; 52 fail, 8 pass).

Eligible pool. ChatDev is excluded by design: its traces run from about 60,000 to 500,000 tokens of cleaned text (median 83,893), too long to code or to check reliably. Traces above 30,000 tokens are also excluded. That leaves 893 MAST-failed and 373 MAST-clean traces across six frameworks: AG2 286 failed, MetaGPT 408, Magentic-One 127, AppWorld 27, OpenManus 27 and HyperAgent 18. Figure 2 shows how often the judge flags each mode by framework. Model is largely confounded with framework: four frameworks ran a single model, and "Claude" carries no version number.

5 Method

Ingest and rendering. Each framework writes a different log format. A parser per format splits each trace into numbered steps, keeps agent text verbatim, and drops only log scaffolding and pure noise (package-installation logs, token counters, repeated web-server warnings). A word-level check confirmed that only scaffolding is lost. Each trace is rendered as a document with numbered steps, and every coder,

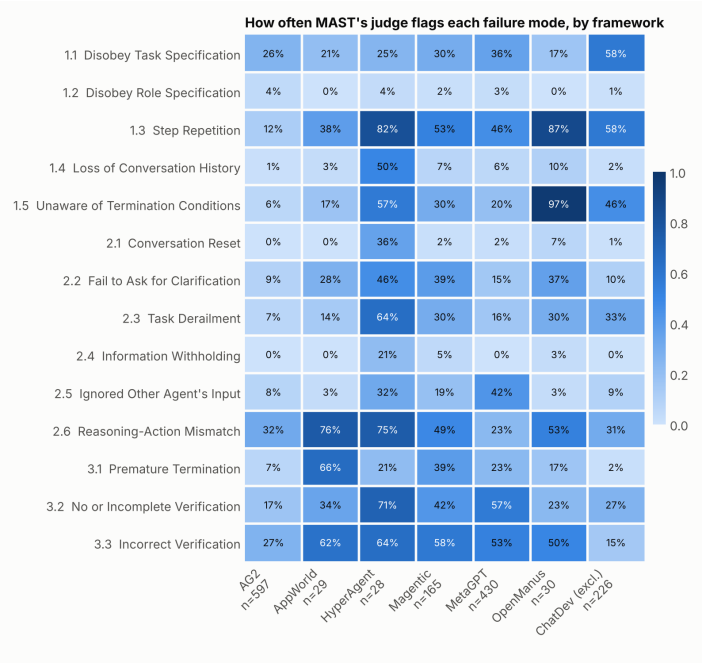


Figure 2: Share of kept traces in which MAST's judge flags each failure mode, by framework. ChatDev is shown for completeness but excluded from the study.

human or AI, reads the same rendering, so that disagreement concerns the application of the codebook and not what each coder saw. MAST's labels are hidden. Information the agents never saw (benchmark answers, gold patches and the benchmark's own pass/fail verdict) is hidden and replaced by a placeholder, which keeps the step numbering stable and prevents the outcome from colouring the coding.

Information audit (RQ2). For each framework, each of the eighteen items in Ezell et al.'s Table 1 is classified as present, partial, absent or not applicable to benchmark runs. Where a reliable automatic check exists (timestamps, tool calls and outputs, reasoning text, model identifiers, sampling settings, package versions, error messages), the share of traces in which it fires is reported, and the status is a judgement informed by that share. Otherwise the classification rests on reading three sampled traces per framework (the first, middle and last by identifier), with the reason recorded for every judgement. All seven frameworks are audited, ChatDev included. What MAST's dataset records outside the logs (framework, an unversioned model name and the benchmark) is noted separately (`docs/information_audit.md`).

AI pilot (RQ3 and RQ4). Claude codes a sample of MAST-failed traces with codebook v1. The primary annotator is `claude-sonnet-5-5`; a second model, `claude-opus-5-5`, codes the same traces. Each annotation runs through Claude Code's headless mode on the author's Claude subscription, with Claude Code's tools, MCP servers and commands switched off and its default system prompt replaced by the annotator prompt (`prompts/annotator_v1.md`) with the codebook pasted in. The trace is the user message, and the reply is constrained by a JSON schema whose allowed codes are read from the codebook, so the annotator cannot invent codes and must give step-referenced evidence for each code, a causal summary, an evidence grade for any violation, and, for each error code, the Ezell et al. cognitive function or functions involved. Every annotation records the backend, the model requested and the model that served it, the prompt and codebook versions, a hash of the full system prompt, token usage and a session identifier. A refusal is recorded as an error and is not routed to another model. Each evidence quote is then checked automatically against the step it cites, after normalising whitespace, quotation marks, capitals and markdown (a quote shortened with an ellipsis must have each piece present, in order). A quote is classed as found in the cited step, found in a different step, not found in any step the annotator could see, or citing a hidden or non-existent step; codes that no evidence item supports are also counted. The check shows how often the annotator's evidence is genuine. It does not show that a genuine quote supports the code it is attached to. At API list prices the 250-trace run would cost about $14 with `claude-sonnet-5-5`; through the subscription it counts against the plan's usage instead.

The sample is the 250-trace bulk sample: MAST-failed traces outside the frozen calibration set (below), balanced across frameworks (AG2 64, AppWorld 23, HyperAgent 13, Magentic-One 63, MetaGPT 64, OpenManus 23) and across models within each framework. Each trace carries a weight equal to the number of eligible traces it represents, so that overall shares describe the eligible pool rather than the balanced sample; they are reported with Wilson 95% intervals on the Kish effective sample size $n_{\text{eff}} = (\sum w)^2 / \sum w^2$. The annotator is offered every trace-codable code but not L4.4 (Section 3.3). Instead, model-level patterns are examined across traces with the framework and benchmark held fixed: GPT-4o against Claude in MetaGPT on ProgramDev, Qwen against CodeLlama in MetaGPT on ProgramDev-v2, and Claude against GPT-4o in AG2 on GSM-Plus. A difference that follows the model suggests, but does not establish, a model-level contribution; the groups are small and the comparisons exploratory.

Consistency measures. Consistency is measured between the two models and between two runs of the primary model on the same traces. Agreement uses Cohen's κ [27], read with the Landis and Koch bands [28], for the presence of each layer, for each code (averaged across codes, never pooled), for error versus violation, for the violation evidence grade, for the deepest layer (with a linearly weighted variant on the ordered scale L1 to L4), and for whether a failure was observed. To compare the two ways of typing an error, the mean per-label κ over HFACS's three error types is set against the mean over Ezell et al.'s four cognitive functions; the schemes differ in number of categories, so the comparison is indicative. the share of identical full code sets is also reported, because exact matches are much rarer than partial ones in automated HFACS coding [20]. Each headline κ carries a 95% bootstrap interval (2,000 resamples), omitted where either coder gave fewer than three positive or three negative answers, because κ behaves badly for rare or near-universal categories [29]. The natural point of comparison is the 0.43 at which MAST's own judge agrees with itself (Section 4). These measures show whether the framework can be applied *consistently*. They do not show that the codes are *correct*: two runs of a model can agree and both be wrong. Correctness requires independent human coders, which is the purpose of the planned validation.

Expert case studies. The author, a flight safety officer, reviews a small number of the AI's codings with the trace alongside. Reviewing a coding is a different task from coding blind, and these reviews are presented as illustrations of what the framework shows, not as validation.

Planned validation (second study). A calibration set of thirty traces was drawn before this study changed direction and has been frozen for human validation. It holds five traces from each of six frameworks; in four frameworks one of the five is a MAST-clean control in place of a failed trace, giving 26 failed traces and 4 blind controls. A development set of 18 traces and a held-out test set of 12 were fixed before any coding, and the hidden key's SHA-256 fingerprint is committed to the repository. Independent coders will code these traces blind in a purpose-built coding tool, giving the step references that support each code rather than quotes (one quote is required only for a violation graded *stated*, so the grade can be checked), first with the current codebook and then, after any revision, in a second review pass, before seeing any AI coding. The AI pilot does not touch these thirty traces, so that they remain uncontaminated for human coders.

6 Results

6.1 RQ1: cross-walk

Every factor in Ezell et al.'s taxonomy has a place in AFACS (Table 2), most of them at L4, L2.1 or the L1 errors. The extension adds three things with no counterpart in their framework: a supervision tier (L3.1–L3.3), coordination between agents (L2.3), and graded violations. In the other direction, their framework contributes a model-development factor, which AFACS adopts as L4.4 with its own evidence requirement (Section 3.3), and named security-related contexts. Whether the added categories matter in practice is an empirical question, taken up in RQ3.

In the pilot the annotator recorded, for each error code, the Ezell function or functions involved. The pairings follow the cross-walk closely: every skill-based error (E1, $n = 56$ traces) was paired with action; decision errors (E2, $n = 177$) with decision in 175 cases, but also with action (61) and understanding (44); perceptual errors (E3, $n = 128$) with understanding (124) and observation (83). This cannot be

Table 6: Number of Ezell et al.'s eighteen information items each framework's logs provide. Applicable excludes items that do not arise in these benchmark runs (e.g. tool information for agents that use no tools).

Framework	Traces	Applicable	Present	Partial	Absent
AppWorld	29	18	8	5	5
Magentic-One	165	17	6	6	5
OpenManus	30	17	6	4	7
HyperAgent	28	17	5	5	7
AG2	597	15	2	5	8
ChatDev	226	14	6	5	3
MetaGPT	430	10	2	3	5

read as support for the cross-walk. The codebook the annotator read (v1, note 5.11) stated the predicted pairing, so the annotator may have followed it rather than tested it, and the two schemes' definitions share examples (a malformed tool call illustrates both Ezell's action execution and HFACS's skill-based error). The prediction has been removed from the codebook for later coding (v1.1). The comparison of how consistently each scheme is applied (Table 8) is unaffected.

6.2 RQ2: information adequacy

Figure 3 shows, for each framework, the status of each of the eighteen items, and Table 6 counts them. The logs are good records of what the agents said and did: the agents' responses and actions are present in all seven frameworks, and the user's task in six (OpenManus prints it only in part). Everything needed to identify or re-run the system that failed is missing. No framework provides system documentation and change logs, system artefacts, or model and system access. System configuration is at best partial: ChatDev logs its sampling settings (temperature 0.2, top-p 1.0) and Magentic-One's installation log reveals its scaffolding package versions, but no framework logs a random seed, and MAST names models without versions. System prompts are logged only by ChatDev. Labelled reasoning appears only in OpenManus and HyperAgent; in the other frameworks the only reasoning visible is what a model chose to write into its messages, which Ezell et al. note may not reflect its actual reasoning. Timestamps are absent from AG2 and Magentic-One.

Only ChatDev records a model identifier inside the log, and it does not always agree with MAST's metadata: one group of 30 traces that MAST labels GPT-4o logs the identifier `ModelType.GPT_45`, so either the metadata or the log is wrong. Without logged identifiers, such discrepancies cannot be detected in the other six frameworks.

What an investigator can establish therefore depends on the software that produced the trace rather than on the failure (Table 6). For AFACS the consequence is uneven observability across layers. Unsafe acts and coordination are well observed, because agent-to-agent messages are logged. Supervision failures are visible as behaviour, for instance an orchestrator that lets an error stand, but without system prompts or role definitions it is often impossible to tell whether the orchestrator was ever instructed to check. Model development (L4.4) cannot be evidenced from these logs at all, which is why AFACS reserves it for investigators with developer records (Section 3.3). These findings are an empirical instance of the reconstruction difficulties Ezell et al. list as an open challenge.

6.3 RQ3: applicability

The primary annotator (`claude-sonnet-5-5`) coded the 250 bulk traces twice and `claude-opus-5-5` coded them once, all with codebook v1 and the same prompt. All 750 codings were valid; there were no refusals or errors. Table 8 gives the agreement. Layer presence was applied consistently: κ 0.63–0.76 between the two models and 0.74–0.87 between the two runs of one model, substantial to almost perfect. Individual codes agreed less (mean per-code κ 0.52 between models, 0.59 between runs), the usual pattern in HFACS coding, and only 24% and 40% of traces received an identical full set of codes. The weakest codes were `L4.3` (no procedure; κ 0.19 and 0.23), which overlaps with `L4.1`; the rare `L1.V?` (about zero); and `L3.2` and `L3.4` (0.32–0.62). For comparison, MAST's own judge agrees with itself at a mean per-mode κ of 0.43 on its flat labels (Section 4). HFACS's three error types and Ezell et al.'s four cognitive functions were applied about equally consistently (0.55 and 0.58 between models; 0.71 and 0.70 between

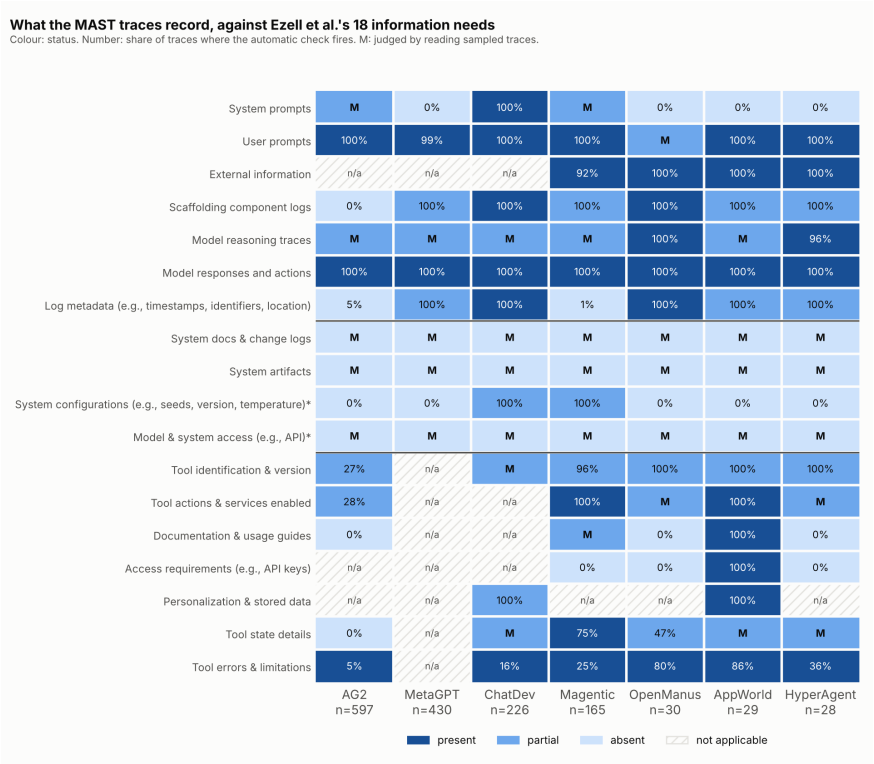


Figure 3: MAST's logs against the eighteen information items of Ezell et al.'s Table 1, by framework. Colour gives the status; numbers give the share of traces in which an automatic check fires; M marks judgements made by reading sampled traces. Items marked * matter mainly for reconstruction.

runs).

The evidence was genuine: of 1,350, 1,328 and 1,350 evidence quotes in the three runs, 99.6%, 99.9% and 99.6% appeared verbatim in the step cited, and none was absent from the trace. The check shows that the words exist, not that they support the code.

Weighted to the eligible pool, the annotator found a failure in 80% of the MAST-failed traces (83% and 79% in the other runs). Among failures, every run coded an unsafe act in all of them, and a contributing factor at L2–L4 in 88% (85%, 91%). The categories AFACS adds to Ezell et al. were common: supervision failures (L3.1–L3.3) in 67% (95% interval 59–75%; other runs 69% and 71%) and coordination breakdowns (L2.3) in 40% (32–48%; 41%, 41%). Violations were a minority: 9% of failures (5–15%; 13%, 10%), of which most rested on the nature of the act (inferred, 6% of failures) rather than the agent's own words (stated, 1%), with 2% indeterminate. They were concentrated in HyperAgent (69% of its failures) and AG2 (24%) and absent from MetaGPT and AppWorld. The typical violation was an agent reporting results of tool calls it never received, or writing "verification" code that only returned a hard-coded answer. Within AG2 on the same Olympiad benchmark, GPT-4o traces carried a violation code in 31% of failures; the groups are too small for a model-level conclusion.

Three features of the coding need care in interpretation. First, design factors (L4) were coded in 63–74% of failures and were the deepest contributing layer in most, but the annotator saw only the trace, so every L4 code is an inference about the system's design; Section 6.4 tests these inferences against the frameworks' documentation. Second, no run left L1 empty for any failure, although the codebook allows a failure with no unsafe act (note 5.8); either such failures are rare in MAST or the annotator is reluctant to use the option, and the prompt's opening instruction to look for "the unsafe acts at the sharp end" may encourage the latter. Third, confidence was "medium" in 86–87% of codings, but it was informative when used: for the 24 codings Sonnet rated "low", its two runs agreed on whether there was a failure in 67% of cases, against 96% for "medium".

Table 7: The annotator's most frequent design claims checked against each framework's documentation and code.

Framework	Claims	Confirmed	Partly true	Contradicted	Not settled
MetaGPT	5	2	3	0	0
AG2	4	1	2	1	0
Magentic-One	6	2	3	1	0
OpenManus	4	2	2	0	0
HyperAgent	3	0	3	0	0
AppWorld	2	1	0	0	1
All	24	8	13	2	1

6.4 Design claims against documentation

Because every L4 code is an inference from the trace, the annotator's most frequent design claims for each framework (from the primary run) were checked against the framework's paper, documentation and source code at the version MAST appears to have run, against MAST's own description, and against framework messages in the logs. MAST publishes no configuration or versions, so the configuration was reconstructed from the logs (`docs/design_check.md` gives every claim, verdict and source). Table 7 summarises the result.

Only two of the 24 claims were wrong outright, but only eight held in full. The typical error was to overstate a weakness as an absence. The annotator wrote "no verifier", "no termination logic" or "no loop detection" where the system has the mechanism in a weak form: MetaGPT's reviewer comments only on the tests, and its runs are capped by rounds and budget; MathChat detects repeated results; Magentic-One's orchestrator asks itself on every turn whether it is in a loop and re-plans when stalled; OpenManus has a duplicate-response detector that only adds a prompt; HyperAgent's editor rejects edits that fail a syntax check, and running tests is optional rather than absent. A weak mechanism that fails leaves the same trace as a missing one, so a trace cannot distinguish them. Two claims placed a failure at the wrong layer: Magentic-One's failed audio transcription is a run-time fault (L2.1) in a design that includes transcription, and OpenManus's missing Python packages come from how the environment was set up. The annotator also missed limits that no trace reveals, such as Magentic-One's 20-turn budget, used almost in full by 56 of its 195 traces. Design factors coded from traces alone should therefore be read as "no effective safeguard visible in the trace"; a later version of the codebook could grade the evidence for L4 codes, as it does for violations, as documented or inferred.

The check also found discrepancies in MAST's own documentation. The paper describes MetaGPT as a system with an explicit verifier, but the logs show MetaGPT's three-role example, in which a reviewer comments only on tests and no agent revises the code (425 of the 430 traces follow its exact sequence). MAST's HyperAgent traces come from a public SWE-bench submission dated September 2024 and are labelled Claude-3.7-Sonnet, a model released later; the submission's own metadata names GPT-4o. The multi-agent AppWorld set-up and the AG2 MMLU team are MAST's own constructions, with prompts unpublished, and the Magentic-One version could be inferred only from line numbers in a traceback. These add to the reconstruction difficulties found in RQ2.

6.5 RQ4: MAST's flat labels

Before coding, the framework implies where each MAST mode should fall (Table 9). The central expectation is that FM-1.1 Disobey Task Specification spreads across layers and across errors and violations, because it can describe a misread task, a deficient specification, a registered requirement that was ignored, or, as in the empty-task traces, no task at all.

Observed against these expectations (Sonnet run 1, weighted shares of failed traces carrying each MAST mode), every MAST mode spread across layers. Within each mode at least 62% of traces had a supervision factor (L3) and between 49% and 100% a precondition (L2); at least half had a design factor (L4) in every mode except FM-2.2 Fail to Ask for Clarification (48%) and the four traces of FM-2.4 Information Withholding. FM-1.1 Disobey Task Specification ($n = 79$) was a mixture as expected: 17% of these traces carry a violation code (22% and 12% in the other runs) and the rest errors only, while a deficient specification (L3.4) was coded in only 6%. The inter-agent modes were mostly, but not only,

Table 8: Consistency of AI coding with AFACS on the 250-trace bulk sample. Cohen's κ with 95% bootstrap intervals (2,000 resamples); intervals are omitted where either coder gave fewer than three positive or negative answers.

Measure	Sonnet vs Opus	Sonnet run 1 vs run 2
L4 present	0.71 (0.62–0.79)	0.74 (0.65–0.82)
L3 present	0.63 (0.53–0.72)	0.74 (0.65–0.82)
L2 present	0.75 (0.67–0.83)	0.76 (0.67–0.83)
L1 present	0.76 (0.65–0.86)	0.87 (0.78–0.94)
Individual codes (mean per-code κ)	0.52 (0.45–0.57)	0.59 (0.53–0.66)
Violation vs error	0.67 (0.51–0.81)	0.68 (0.50–0.83)
Violation evidence grade	0.56	0.62
Error type, HFACS E1–E3 (mean per-code κ)	0.55	0.71
Error type, Ezell et al. functions (mean per-label κ)	0.58	0.70
Deepest layer (weighted)	0.75 (0.68–0.82)	0.80 (0.73–0.86)
Failure observed	0.69 (0.57–0.79)	0.80 (0.70–0.88)
Identical full code set (share of traces)	24%	40%
Evidence quotes found in the cited step	Sonnet 99.6%, 99.6%; Opus 99.9%	

Table 9: Where the framework expects MAST's modes to fall in AFACS, to be tested against the AI pilot.

MAST mode	Expected AFACS location
1.1 Disobey task specification	Spread: L3.4, L1.E2 or L1.E3, L1.V, or L4.5 (a run set up with no task)
1.2 Disobey role specification	L4.1 role design, or L1.E2 / L1.V
1.3 Step repetition	L1 act, with L3.3 where a supervisor could have broken the loop, and L4.1 (no termination logic)
1.4 Loss of conversation history	L2.2 agent state, with L4.2 context limits
1.5 Unaware of termination conditions	L4.1 termination logic, or L1.E2
2.1–2.5 Inter-agent misalignment	Mostly L2.3 coordination; 2.2 also L3.4
2.6 Reasoning–action mismatch	L1.E1 (sound intention, faulty execution), or L1.V
3.1 Premature termination	L1.E2, or L1.V (false claim of completion), with L3.1
3.2 No or incomplete verification	By whose job it was: L1 (incl. L1.V1 routine skipping), L3.1, or L4.1/L4.3
3.3 Incorrect verification	L3.1, L1.E2 or L1.E3

coordination: L2.3 appears in 52% of FM-2.3 Task Derailment and 62% of FM-2.5 Ignored Other Agent's Input. FM-1.3 Step Repetition ($n = 118$) was coded as a design factor (L4.1, 60%) more often than as a supervisor failing to break the loop (L3.3, 27%). FM-3.2 No or Incomplete Verification ($n = 101$) split between supervision (L3.1, 69%) and design (L4.1, 69%), and was never coded as a routine violation. A single flat label therefore covers failures whose remedies differ: a missing verifier, a supervisor who did not check, and an agent that departed from a requirement it had registered.

7 Case Studies

[Pending: three or four traces from the bulk sample, reviewed by the author, where layered coding changes the account of the failure].

One illustration of the flat-label problem comes from the audit. In `MetaGPT_ProgramDev2_GPT4o__099` the task text is empty: the harness passed nothing (step S2). The coder agent writes the placeholder `your_code_here` (S3). The tester writes template tests for a function that does not exist (S4). The reviewer asks for more specific test inputs and never mentions that there is no task and no code (S5), and the exchange repeats once more before the run ends (S6–S7). MAST labels the trace FM-1.1 Disobey Task Specification and FM-2.2 Fail to Ask for Clarification. There was no specification to disobey. Under the violation rule, condition (1) cannot hold, so none of the agents' behaviour here can be coded as a violation. The deepest factor is the run's set-up, which AFACS codes as L4.5 Operation set-up; whether the agents' failure to notice the missing task is also an unsafe act is a question for the coding, not something the label settles.

8 Discussion

The information audit already has an implication for how multi-agent systems are logged. Ezell et al. recommend that developers retain system documentation, exact versions and runtime settings for investigators; MAST's traces, which are among the most widely used public records of multi-agent failure, contain almost none of them. A logging standard for multi-agent research releases, covering at least model identifiers and versions, sampling settings and seeds, system prompts and role definitions, and timestamps, would let later studies reconstruct the failures they analyse.

The pilot suggests that the extension earns its place. The categories with no counterpart in Ezell et al.'s framework were among the most frequently coded: supervision failures in about two-thirds of failures, coordination in about two-fifths. Violations were rarer, but they were where MAST's labels were least informative, and the evidence grade showed that most rested on the act rather than on stated reasoning. The layers could be applied consistently by an AI annotator, at a level well above MAST's own judge. Two cautions temper this. Consistency is not correctness: two runs of a model can share a mistake, and only independent human coding can show which codes are right. And the deeper the layer, the less a trace alone can establish: design factors were the most common deepest layer, yet they are the least observable in a trace, which is why the documentation check (Section 6.4) matters. In aviation, organisational findings come from company records rather than the flight recorder; the same division of evidence applies here.

9 Limitations

- **No human validation yet.** The pilot measures consistency, not correctness; agreement between AI runs can coexist with shared error. Human validation is the next study.
- **AI coding AI traces.** The annotator is a language model coding the behaviour of other language models, and may share their blind spots.
- **Design factors inferred from traces.** The annotator saw only the trace, so its L4 codes are inferences about each system's design. Checked against documentation (Section 6.4), its most frequent design claims were usually right about the weakness but overstated it as an absence; establishing design factors properly needs reviews that combine the trace with the system's documentation, which future case-study work should provide.

- **No failure without an unsafe act.** Although the codebook allows it, the annotator never coded a failure without an unsafe act. Whether such failures are rare or the annotator (or its prompt's framing) avoids the option needs further research.
- **Outcomes hidden from the annotator.** To avoid hindsight bias, benchmark answers and verdicts were hidden. The annotator therefore could not tell whether, for example, a mathematical answer was wrong, which probably explains most of the AG2 traces it judged failure-free.
- **The Ezell pairing is not a test.** The codebook the annotator read stated the predicted pairing of error types and cognitive functions (Section 6), so the observed pairing cannot support the cross-walk. The prediction is removed in codebook v1.1.
- **Confidence rarely varied.** Most codings were rated "medium", so confidence can flag only a minority of uncertain codings, although those it did flag were less stable between runs.
- **Behavioural, not intentional.** "Violation" describes what the trace shows. It is not a claim about what a model wanted, and stated reasoning may not reflect what produced an action.
- **Annotation route.** Annotations run through Claude Code's headless mode rather than the bare API. Tools are disabled and the system prompt replaced, but the surrounding harness is Claude Code's, and results with the same prompt through the API may differ slightly.
- **Noisy reference labels.** MAST's labels come from an LLM judge with a test–retest κ of 0.43 on the duplicated traces, a small sample of 30.
- **No ground-truth outcome.** "Failed" means flagged by MAST's judge, except for the 60 traces that carry a benchmark verdict.
- **Failures, not incidents.** Benchmark failures carry no harm; the occurrence-reporting analogy is an argument, not a demonstration.
- **Coverage and confounding.** ChatDev and traces above 30,000 tokens are excluded. Model is confounded with framework and, within AG2, with benchmark, so model comparisons are exploratory.
- **Contested reliability.** HFACS reliability is itself debated [17, 18].

10 Future Work

The next study validates AFACS with independent human coders on the frozen calibration set, measuring agreement between humans and between humans and the AI annotator on the pre-registered held-out traces. Three questions from the pilot belong there. Case-study reviews that combine each trace with the system's documentation should establish how often trace-inferred design factors are right. A variant prompt without the instruction to look for unsafe acts, applied to a subset of traces, would show whether the absence of failures without an unsafe act is real or induced. And giving the annotator the outcome (the benchmark's verdict, though not the answer), much as an investigator knows that an accident occurred, or using datasets with verified outcomes and human attribution of the responsible agent and step, such as Who&When [14], would remove the annotator's blindness to whether a task actually failed.

A conceptual question is left open. AFACS places the user who sets the task in the supervision tier and the people who launch runs in the organisational tier (Section 3.1). A fifth level above the organisation, covering users, operators and the regulatory context, may describe deployed systems better, where the people around the system are many and their decisions are visible in logs. It was not adopted here, to keep the direct correspondence with HFACS.

A further phase applies AFACS to a domain evaluation of aviation dispatch and safety decisions under pressure. Each scenario has a ground-truth answer (a minimum equipment list item, a flight-time limitation, weather minima or a fuel decision) and is run in neutral, commercial-pressure, authority-pressure and user-frustration variants. The measure is how often the safe answer survives the pressure, and the failures are coded with AFACS, with particular attention to error versus violation. It would be built in the UK AI Security Institute's Inspect framework [30] from public regulatory sources only. Related aviation evaluations include Pre-Flight, a benchmark of aviation operational knowledge available in Inspect [31]; PilotBench, which tests trajectory and attitude prediction under safety constraints [32]; and AeroCopilotBench, which evaluates agents on aircraft emergency procedures in an executable cockpit [33]. None of these codes the failures it finds with a human-factors framework.

Use of AI Tools

The data pipeline, tests and figures were built with Claude Code under the author's direction. The AI pilot's annotations are produced by Claude models through Claude Code's headless mode on the author's subscription, as described in Section 5. The framework, the codebook decisions and the expert reviews are the author's. Every study-design decision is recorded with its reason in `src/afacs/design.py`, and the pipeline is open.

Data and Code

MAST dataset: Cemri et al. [8], CC-BY-4.0. Code: MIT licence. Traces are not redistributed because they contain benchmark questions whose maintainers ask that they not be reposted in crawlable form; the pipeline recreates them from the pinned dataset revision. The AI codings are committed with their codes in plain text and their quotes and summaries encoded, with a canary string, so that benchmark text is not exposed to web crawlers; one command decodes them.

References

[1] J. Reason, *Human Error*. Cambridge University Press, 1990.

[2] J. Reason, *Managing the Risks of Organizational Accidents*. Ashgate, 1997.

[3] S. A. Shappell and D. A. Wiegmann, "The Human Factors Analysis and Classification System–HFACS," U.S. Department of Transportation, Federal Aviation Administration, Office of Aviation Medicine, Report DOT/FAA/AM-00/7, 2000.

[4] D. A. Wiegmann and S. A. Shappell, *A Human Error Approach to Aviation Accident Analysis: The Human Factors Analysis and Classification System*. Ashgate, 2003.

[5] J. Reason, A. Manstead, S. Stradling, J. Baxter and K. Campbell, "Errors and violations on the roads: a real distinction?" *Ergonomics*, vol. 33, no. 10–11, pp. 1315–1332, 1990.

[6] C. Ezell, X. Roberts-Gaal and A. Chan, "Incident analysis for AI agents," in *Proc. AAAI/ACM Conference on AI, Ethics, and Society (AIES)*, 2025. arXiv:2508.14231.

[7] B. Baker, J. Huizinga, L. Gao, Z. Dou, M. Y. Guan, A. Madry, W. Zaremba, J. Pachocki and D. Farhi, "Monitoring reasoning models for misbehavior and the risks of promoting obfuscation," arXiv:2503.11926, 2025.

[8] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez and I. Stoica, "Why do multi-agent LLM systems fail?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2503.13657.

[9] M. Shamsujjoha, Q. Lu, D. Zhao and L. Zhu, "Swiss cheese model for AI safety: A taxonomy and reference architecture for multi-layered guardrails of foundation model based agents," arXiv:2408.02205, 2024.

[10] S. Mylius, "Systematic hazard analysis for frontier AI using STPA," arXiv:2506.01782, 2025.

[11] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2012.

[12] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan and R. Qian, "TRAIL: Trace reasoning and agentic issue localization," arXiv:2505.08638, 2025.

[13] K. Zhu et al., "Where LLM agents fail and how they can learn from failures," arXiv:2509.25370, 2025.

[14] S. Zhang et al., "Which agent causes task failures and when? On automated failure attribution of LLM multi-agent systems," in *Proc. ICML*, PMLR vol. 267, pp. 76583–76599, 2025.

[15] H. Raj, V. Gupta, A. Mahmoud, R.-G. Dumitru, D. Yi, A. Sabharwal and Y. He, "Model or harness? An interaction-centric taxonomy for localizing agent failures," arXiv:2607.28802, 2026.

[16] D. A. Wiegmann and S. A. Shappell, "A human error analysis of commercial aviation accidents using the Human Factors Analysis and Classification System (HFACS)," U.S. Department of Transportation, Federal Aviation Administration, Office of Aviation Medicine, Report DOT/FAA/AM-01/3, 2001.

[17] N. S. Olsen and S. T. Shorrock, "Evaluation of the HFACS-ADF safety classification system: Inter-coder consensus and intra-coder consistency," *Accident Analysis & Prevention*, vol. 42, no. 2, pp. 437–444, 2010.

[18] T. N. Cohen, D. A. Wiegmann and S. A. Shappell, "Evaluating the reliability of the Human Factors Analysis and Classification System," *Aerospace Medicine and Human Performance*, vol. 86, no. 8, pp. 728–735, 2015.

[19] B. R. Burnham, "Letter to the Editor re: Evaluating the reliability of the Human Factors Analysis Classification System," *Aerospace Medicine and Human Performance*, vol. 88, no. 6, p. 597, 2017.

[20] A. Ahmadi, S. Sharif and Y. Banad, "Improving aviation safety analysis: Automated HFACS classification using reinforcement learning with group relative policy optimization," arXiv:2508.21201, 2025.

[21] V. Krakovna, J. Uesato, V. Mikulik, M. Rahtz, T. Everitt, R. Kumar, Z. Kenton, J. Leike and S. Legg, "Specification gaming: the flip side of AI ingenuity," DeepMind blog, April 2020.

[22] Z. Zhong, A. Raghunathan and N. Carlini, "ImpossibleBench: Measuring LLMs' propensity of exploiting test cases," arXiv:2510.20270, 2025.

[23] A. Meinke, B. Schoen, J. Scheurer, M. Balesni, R. Shah and M. Hobbhahn, "Frontier models are capable of in-context scheming," arXiv:2412.04984, 2024.

[24] A. Lynch, B. Wright, C. Larson, S. J. Ritchie, S. Mindermann, E. Hubinger, E. Perez and K. Troy, "Agentic misalignment: How LLMs could be insider threats," arXiv:2510.05179, 2025.

[25] N. Smyth et al., "Quantifying overclaiming propensity in frontier LLM agents," arXiv:2609.20812, 2026.

[26] A. Hägele, A. P. Gema, H. Sleight, E. Perez and J. Sohl-Dickstein, "The hot mess of AI: How does misalignment scale with model intelligence and task complexity?" in *Proc. ICLR*, 2026. arXiv:2601.23045.

[27] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[28] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[29] A. R. Feinstein and D. V. Cicchetti, "High agreement but low kappa: I. The problems of two paradoxes," *Journal of Clinical Epidemiology*, vol. 43, no. 6, pp. 543–549, 1990.

[30] UK AI Security Institute, "Inspect AI: Framework for large language model evaluations," software, 2024. `https://inspect.aisi.org.uk/`.

[31] A. Brooker and T. Hughes, "Pre-Flight: A benchmark for evaluating large language models on aviation operational knowledge," arXiv:2607.01829, 2026.

[32] Y. Wu, H. Liu, Z. Li and B. Wang, "PilotBench: A benchmark for general aviation agents with safety constraints," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2026. arXiv:2604.08987.

[33] Y. Yuan, Z. Wu, Y. Li, L. Ma and K. Li, "AeroCopilotBench: Safety-gated evaluation of LLM agents on aircraft emergency procedures in an executable cockpit," arXiv:2608.16349, 2026.